

Python Coding Challenge

Problem 1: Smart Electricity Billing System

Problem Statement

Monthly electricity consumption (units) of different houses in a residential society is stored as follows:

Sample Data

```
units = {  
    "House101": 320,  
    "House102": 180,  
    "House103": 510,  
    "House104": 275,  
    "House105": 150,  
    "House106": 430,  
    "House107": 220,  
    "House108": 390,  
    "House109": 145,  
    "House110": 600  
}
```

Tasks

1. Display houses consuming more than 400 units.
2. Find the highest-consuming house.
3. Find the lowest-consuming house.
4. Calculate the total units consumed.
5. Create separate lists for:
 - Low Consumption (< 200)
 - Medium Consumption (200–400)
 - High Consumption (> 400)
6. Count houses eligible for an energy-saving campaign (consumption > 300).

Sample Output

Houses Consuming More Than 400 Units:

House103
House106
House110

Highest Consumption:

House110 (600 units)

Lowest Consumption:

House109 (145 units)

Total Units Consumed: 3220

Low Consumption:

['House102', 'House105', 'House109']

Medium Consumption:

['House101', 'House104', 'House107', 'House108']

High Consumption:

['House103', 'House106', 'House110']

Eligible for Energy-Saving Campaign: 5

Problem 2: Student Scholarship Evaluation System

Problem Statement

The marks obtained by students in the final examination are stored as follows:

Sample Data

```
marks = {  
    "Anuj": 92,  
    "Rahul": 76,  
    "Priya": 88,  
    "Neha": 64,  
    "Amit": 58,  
    "Sneha": 95,  
    "Karan": 81,  
    "Pooja": 73,  
    "Rohit": 47,  
    "Anjali": 90  
}
```

Tasks

1. Display students scoring above 85 marks.
2. Find the topper.
3. Find the student with the lowest marks.
4. Calculate class average marks.
5. Generate grades:
 - A (90+)
 - B (75–89)
 - C (50–74)
 - F (<50)
6. Create a list of scholarship students (marks $\geq$ 90).

Sample Output

Students Scoring Above 85:

Anuj
Priya
Sneha

Anjali

Topper:

Sneha (95)

Lowest Scorer:

Rohit (47)

Average Marks: 76.4

Scholarship Students:

['Anuj', 'Sneha', 'Anjali']

Problem 3: Food Delivery Performance Dashboard

Problem Statement

Delivery times (in minutes) for different orders are recorded below:

Sample Data

delivery_times = [28, 45, 60, 22, 35, 80, 40, 25, 55, 18]

Tasks

1. Find the fastest delivery time.
2. Find the slowest delivery time.
3. Calculate the average delivery time.
4. Display delayed orders (>45 minutes).
5. Categorize deliveries:
 - Fast (≤ 30 minutes)
 - Normal (31–45 minutes)
 - Delayed (>45 minutes)

Sample Output

Fastest Delivery: 18 minutes

Slowest Delivery: 80 minutes

Average Delivery Time: 40.8 minutes

Delayed Orders:

[60, 80, 55]

Fast Deliveries: 4

Normal Deliveries: 3

Delayed Deliveries: 3

Problem 4: Online Shopping Inventory System

Problem Statement

An online store maintains stock quantities of products.

Sample Data

```
inventory = {  
    "Laptop": 15,  
    "Mouse": 45,  
    "Keyboard": 32,  
    "Monitor": 12,  
    "Headphones": 28,  
    "Printer": 8,  
    "Webcam": 20,  
    "Speaker": 18,  
    "Tablet": 10,  
    "Router": 25  
}
```

Tasks

1. Display products with stock below 15 units.
2. Find the product with maximum stock.
3. Find the product with minimum stock.
4. Calculate total stock available.
5. Create a list of products requiring restocking (<10 units).

Sample Output

Products with Stock Below 15:

Monitor

Printer

Tablet

Highest Stock Product:

Mouse (45 units)

Lowest Stock Product:

Printer (8 units)

Total Stock Available: 213

Products Requiring Restocking:

['Printer']

Problem 5: Mobile Screen Time Analyzer

Problem Statement

Daily mobile screen time (in minutes) of a student is recorded for 10 days.

Sample Data

```
screen_time = [180, 220, 150, 300, 120, 250, 190, 210, 175, 260]
```

Tasks

1. Calculate average screen time.
2. Find the highest and lowest screen time.
3. Count days exceeding 200 minutes.
4. Display days with healthy usage (<180 minutes).
5. Categorize usage:
 - Healthy (<180)
 - Moderate (180–240)
 - Excessive (>240)

Sample Output

Average Screen Time: 205.5 minutes

Highest Screen Time: 300 minutes

Lowest Screen Time: 120 minutes

Days Exceeding 200 Minutes: 5

Healthy Usage Days:

Day 3

Day 5

Day 9

Healthy: 3

Moderate: 4

Excessive: 3

Problem 6: Library Book Availability System

Problem Statement

The number of available copies of books in a library is stored below.

Sample Data

```
books = {  
    "Python": 5,  
    "Java": 2,  
    "DBMS": 4,  
    "Networking": 1,  
    "OS": 3,  
    "AI": 6,  
    "ML": 2,  
    "Cloud": 5,  
    "Cyber Security": 1,  
    "Web Development": 4  
}
```

Tasks

1. Display books with fewer than 3 copies.
2. Find the book with maximum copies.
3. Find the book with minimum copies.
4. Count total books available.
5. Generate a restocking list.

Sample Output

Books Requiring Attention:

Java

Networking

ML

Cyber Security

Book with Maximum Copies:

AI (6 copies)

Book with Minimum Copies:

Networking (1 copy)

Total Copies Available: 33

Restocking List:

['Java', 'Networking', 'ML', 'Cyber Security']

Problem 7: Cricket Tournament Statistics

Problem Statement

Runs scored by players in a tournament are given below.

Sample Data

```
runs = {  
    "Virat": 645,  
    "Rohit": 512,  
    "Gill": 698,  
    "Rahul": 435,  
    "Hardik": 278,  
    "Pant": 534,  
    "Surya": 389,  
    "Jadeja": 301,  
    "Iyer": 455,  
    "KL": 410  
}
```

Tasks

1. Find the Orange Cap winner.
2. Find the lowest scorer.

3. Calculate total runs scored.
4. Display players scoring more than 500 runs.
5. Create a list of players scoring below 400.

Sample Output

Orange Cap Winner:

Gill (698 runs)

Lowest Scorer:

Hardik (278 runs)

Total Runs: 4657

Players Scoring Above 500:

Virat

Rohit

Gill

Pant

Players Scoring Below 400:

['Hardik', 'Surya', 'Jadeja']

Problem 8: City Temperature Monitoring System

Problem Statement

Daily temperatures of different cities are stored below.

Sample Data

```
temperature = {  
    "Delhi": 41,  
    "Mumbai": 33,  
    "Chennai": 37,  
    "Kolkata": 39,  
    "Bengaluru": 28,  
    "Pune": 30,  
    "Jaipur": 42,  
    "Lucknow": 40,  
    "Hyderabad": 35,  
    "Ahmedabad": 43  
}
```

Tasks

1. Display cities with temperature above 40°C.
2. Find the hottest city.
3. Find the coolest city.
4. Calculate average temperature.
5. Create a list of pleasant cities (<35°C).

Sample Output

Cities Above 40°C:

Delhi

Jaipur

Ahmedabad

Hottest City:

Ahmedabad (43°C)

Coollest City:

Bengaluru (28°C)

Average Temperature: 36.8°C

Pleasant Cities:

['Mumbai', 'Bengaluru', 'Pune']

Problem 9: Hospital Patient Monitoring System

Problem Statement

Patient heart rates are recorded below.

Sample Data

```
heart_rate = {  
    "P101": 72,  
    "P102": 105,  
    "P103": 88,  
    "P104": 120,  
    "P105": 65,  
    "P106": 98,  
    "P107": 110,  
    "P108": 70,  
    "P109": 85,  
    "P110": 130  
}
```

Tasks

1. Display critical patients (heart rate >100).
2. Find highest and lowest heart rate.
3. Calculate average heart rate.
4. Count stable patients (60–100 bpm).

Sample Output

Critical Patients:

P102

P104

P107

P110

Highest Heart Rate:

P110 (130 bpm)

Lowest Heart Rate:

P105 (65 bpm)

Average Heart Rate: 94.3 bpm

Stable Patients: 6

Problem 10: Movie Rating Analysis System

Problem Statement

Ratings given by users for movies are stored below.

Sample Data

```
ratings = {  
    "Inception": 4.8,  
    "Avatar": 4.3,  
    "Titanic": 4.5,  
    "Joker": 4.7,  
    "Frozen": 3.8,  
    "Interstellar": 4.9,  
    "Dune": 4.6,  
    "Up": 4.1,  
    "Coco": 4.4,  
    "Cars": 3.9  
}
```

Tasks

1. Display movies rated above 4.5.
2. Find the highest-rated movie.
3. Find the lowest-rated movie.
4. Calculate average rating.
5. Create a recommendation list (rating $\geq$ 4.5).

Sample Output

Movies Rated Above 4.5:

Inception
Joker
Interstellar
Dune

Highest Rated Movie:

Interstellar (4.9)

Lowest Rated Movie:

Frozen (3.8)

Average Rating: 4.4

Recommended Movies:

['Inception', 'Titanic', 'Joker', 'Interstellar', 'Dune']

Problem 11: Student Feedback Analysis System

Problem Statement

A training institute collects feedback from students after completing a Python course. The feedback comments are stored in a text file named feedback.txt.

Sample Input/Data (feedback.txt)

The sessions were very interactive and informative.
Excellent teaching methodology and practical examples.
The pace of the course was appropriate.
More real-world projects should be included.
The trainer explained concepts very clearly.

Tasks

1. Count the total number of lines.
2. Count the total number of words.
3. Count the total number of characters.
4. Find the longest feedback comment.
5. Find the shortest feedback comment.
6. Count the total number of vowels present in the file.

Sample Output

Total Lines: 5

Total Words: 35

Total Characters: 220

Longest Feedback:
Excellent teaching methodology and practical examples.

Shortest Feedback:
The pace of the course was appropriate.

Total Vowels: 76

Problem 12: Employee Salary Report Generator

Problem Statement

Employee details are stored in a text file named employees.txt.

Sample Input/Data (employees.txt)

EMP101,Anuj,45000
EMP102,Rahul,52000
EMP103,Priya,38000
EMP104,Neha,61000
EMP105,Amit,29000
EMP106,Sneha,55000
EMP107,Karan,47000
EMP108,Pooja,72000
EMP109,Rohit,33000
EMP110,Anjali,68000

Tasks

1. Display employees earning more than ₹50,000.
2. Find the highest-paid employee.
3. Find the lowest-paid employee.
4. Calculate the average salary.
5. Generate salary categories:
 - High ($\geq$ ₹60,000)
 - Medium (₹40,000 – ₹59,999)
 - Low ($<$ ₹40,000)

Sample Output

Employees Earning Above ₹50,000:

Rahul
Neha
Sneha
Pooja
Anjali

Highest Paid Employee:

Pooja (₹72,000)

Lowest Paid Employee:

Amit (₹29,000)

Average Salary: ₹50,000

High Salary:

['Neha', 'Pooja', 'Anjali']

Medium Salary:

['Anuj', 'Rahul', 'Sneha', 'Karan']

Low Salary:

['Priya', 'Amit', 'Rohit']

Problem 13: Library Book Issue Tracker

Problem Statement

A library stores the number of times books were issued during a month.

Sample Data

book_issues = [15, 8, 22, 10, 18, 5, 30, 12, 20, 25]

Tasks

1. Find the maximum number of issues.
2. Find the minimum number of issues.
3. Calculate the average number of issues.
4. Count books issued more than 15 times.
5. Create a list of books issued fewer than 10 times.

Sample Output

Maximum Issues: 30

Minimum Issues: 5

Average Issues: 16.5

Books Issued More Than 15 Times: 5

Books Issued Fewer Than 10 Times:
[8, 5]

Problem 14: Shopping Cart Billing System

Problem Statement

The prices of products purchased by a customer are stored in a tuple.

Sample Data

prices = (1250, 799, 450, 999, 300, 1500, 650, 250, 850, 1200)

Tasks

1. Calculate the total bill amount.
2. Find the most expensive product.
3. Find the least expensive product.
4. Count products costing more than ₹1,000.
5. Create a list of products eligible for discount (price > ₹800).

Sample Output

Total Bill Amount: ₹8,248

Most Expensive Product: ₹1,500

Least Expensive Product: ₹250

Products Costing More Than ₹1,000: 3

Discount Eligible Products:
[1250, 999, 1500, 850, 1200]

Problem 15: Function-Based Temperature Converter

Problem Statement

Daily temperatures recorded in Celsius are given below.

Sample Data

temperatures = [25, 30, 35, 40, 28, 32, 38, 22, 27, 31]

Tasks

Create functions to:

1. Convert Celsius to Fahrenheit.
2. Display all temperatures in Fahrenheit.
3. Find the highest Fahrenheit temperature.
4. Find the lowest Fahrenheit temperature.
5. Calculate the average Fahrenheit temperature.

Sample Output

Temperatures in Fahrenheit:

77.0
86.0
95.0
104.0
82.4
89.6
100.4
71.6
80.6
87.8

Highest Temperature: 104.0°F

Lowest Temperature: 71.6°F

Average Temperature: 87.14°F

Problem 16: File Copy Utility

Problem Statement

A company wants to maintain backups of important documents. Create a program to copy the contents of one file into another.

Sample Input/Data

Source File (notes.txt)

Python supports file handling.

Functions improve code reusability.

Dictionaries store data in key-value pairs.

Tasks

1. Read the contents of the source file.
2. Copy the entire content to another file named backup.txt.
3. Display a success message.
4. Verify whether both files contain the same number of lines.

Sample Output

File copied successfully.

Source File Lines: 3

Backup File Lines: 3

Verification Status: Successful

Problem 17: Daily Sales Performance Analyzer

Problem Statement

Daily sales figures (in ₹) for 10 days are stored in a list.

Sample Data

```
sales = [15000, 22000, 18000, 25000, 30000, 17000, 28000, 26000, 21000, 19000]
```

Tasks

1. Find the highest sales.
2. Find the lowest sales.
3. Calculate average sales.
4. Count days with sales above ₹20,000.
5. Display sales figures below average.

Sample Output

Highest Sales: ₹30,000

Lowest Sales: ₹15,000

Average Sales: ₹22,100

Days with Sales Above ₹20,000: 5

Sales Below Average:
[15000, 18000, 17000, 21000, 19000]

Problem 18: Student Attendance Percentage Calculator

Problem Statement

The attendance status of a student for 15 days is represented as follows:

Sample Data

attendance = ('P', 'P', 'A', 'P', 'P', 'P', 'A', 'A', 'P', 'P', 'P', 'P', 'A', 'P', 'P')

Tasks

1. Count present days.
2. Count absent days.
3. Calculate attendance percentage.
4. Determine whether attendance is below 75%.
5. Display the attendance status.

Sample Output

Present Days: 11

Absent Days: 4

Attendance Percentage: 73.33%

Attendance Status:
Below 75%

Problem 19: Word Frequency Analyzer

Problem Statement

A text file contains the following paragraph.

Sample Input/Data (article.txt)

Python is easy to learn.
Python is powerful.
Python supports multiple programming paradigms.
Programming with Python is enjoyable.

Tasks

1. Count the total number of words.
2. Count the frequency of each word.
3. Find the most frequently occurring word.
4. Display words appearing only once.
5. Display all unique words.

Sample Output

Total Words: 16

Most Frequent Word:
Python (4 times)

Words Appearing Once:

easy
to
learn
powerful
supports
multiple
paradigms
with
enjoyable

Unique Words Count: 12

Problem 20: Employee Performance Evaluation System

Problem Statement

Employee performance scores are stored below.

Sample Data

```
performance = {  
    "EMP101": 92,  
    "EMP102": 78,  
    "EMP103": 45,  
    "EMP104": 88,  
    "EMP105": 97,  
    "EMP106": 56,  
    "EMP107": 81,  
    "EMP108": 64,  
    "EMP109": 39,  
    "EMP110": 73  
}
```

Tasks

1. Display employees scoring above 80.
2. Count employees needing improvement (score < 60).
3. Find the top performer.
4. Calculate average performance score.
5. Categorize employees:
 - Excellent (≥ 90)
 - Good (75–89)

- Average (60–74)
- Poor (< 60)

Sample Output

Employees Scoring Above 80:

EMP101

EMP104

EMP105

EMP107

Employees Needing Improvement: 3

Top Performer:

EMP105 (97)

Average Score: 71.3

Excellent:

['EMP101', 'EMP105']

Good:

['EMP102', 'EMP104', 'EMP107']

Average:

['EMP108', 'EMP110']

Poor:

['EMP103', 'EMP106', 'EMP109']
